

Heuristic Search

Lecture 6

CS 4880

Kirk Duran

Today

- ▶ Why uninformed search wastes effort when the state space is large.
- ▶ How a heuristic function estimates remaining cost to the goal.
- ▶ How Greedy Best-First Search uses only heuristic guidance.
- ▶ How A* combines path cost and heuristic guidance.
- ▶ Why admissibility and consistency matter for A*.
- ▶ How Weighted A* and related variants trade solution quality for search effort.

Key idea

Informed search uses problem-specific estimates to decide which frontier states appear most promising.

Part 1: From Uninformed Search to Informed Search



Last Time: Breadth-First Search

Unit-cost search

- ▶ BFS expands states in increasing depth.
 - ▶ With unit action costs, minimum depth is minimum path cost.
 - ▶ A FIFO frontier is sufficient because every step contributes the same cost.
-
- ▶ In the source example, BFS reaches the goal using three edges.
 - ▶ The search ignores edge-specific costs because the model treats every action equally.

Image placeholder

BFS table from A to Z with queue, visited set, and reconstructed path.

Key idea

BFS is optimal only for the objective it represents: minimizing the number of unit-cost actions.

Last Time: Dijkstra / Uniform-Cost Search

Cost-aware frontier

- ▶ Dijkstra / UCS expands the frontier state with minimum cumulative path cost.
 - ▶ Priority is determined by $g(n)$.
 - ▶ Edge relaxation replaces an expensive route when a cheaper route is discovered.
-
- ▶ The source example reaches Z with total path cost 17.
 - ▶ Search is cost-aware, but it still has no estimate of which direction leads toward the goal.

Image placeholder

Dijkstra table from A to Z showing g values, priority queue, and parent updates.

Key idea

Uniform-Cost Search knows how expensive the path has been, but not how promising the remaining path appears.

A Human Would Look Toward the Goal

Suppose we are navigating a city and trying to reach a visible landmark.

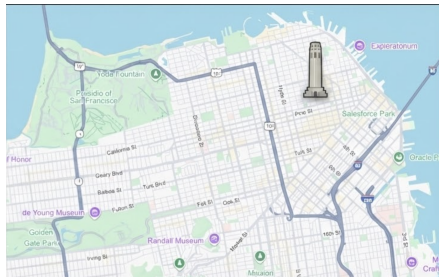
- ▶ BFS can systematically enumerate nearby intersections.
- ▶ Dijkstra can account for travel cost along roads.
- ▶ A person can also use the visible landmark to estimate which direction is promising.

Useful extra information

The landmark provides an estimate of how far we still appear to be from the goal.

Key idea

Search can exploit information that is not encoded by graph connectivity alone.



The Search Problem Solver Problem

Uninformed search uses only the structure of the problem space.

- ▶ BFS, DFS, and iterative deepening do not use problem-specific estimates of progress.
- ▶ In deep or highly branching spaces, large amounts of work may be spent in irrelevant regions.
- ▶ Even simple puzzle domains can contain enormous state spaces.

Example

The 15-puzzle has more than 20 trillion reachable states.



Key idea

Systematic search is not enough by itself; large problems require a way to prioritize promising states.

Cognitive Psychology Meets AI

The source lecture motivates heuristics using observations from human problem solving.

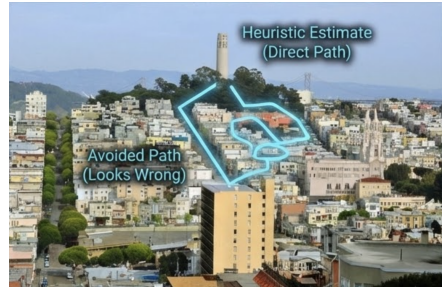
- ▶ People estimate how far they appear to be from a goal.
- ▶ People often avoid paths that appear to move in the wrong direction.
- ▶ Human problem solving is therefore not purely blind enumeration.

AI question

How can we encode “how close does this state appear to be to the goal?”

Key idea

A heuristic converts domain knowledge into a numerical estimate that can guide search order.



Part 2: Heuristic Functions — Estimating Cost-to-Go



Heuristic Function

Definition

A **heuristic function** $h(n)$ estimates the cost of a cheapest path from state n to a goal state.

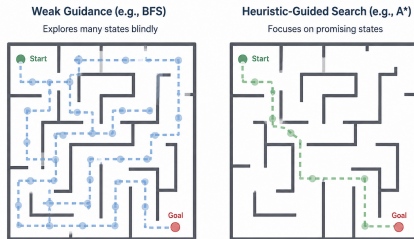
- ▶ Smaller $h(n)$ means the state appears closer to the goal.
- ▶ Heuristics are typically designed for a particular problem domain.
- ▶ A good heuristic can reduce the number of states that must be expanded.

Interpretation

$h(n)$ predicts future cost; it does not measure the cost already incurred.

Key idea

The purpose of a heuristic is not to solve the problem directly, but to rank frontier states more intelligently.



Common Heuristics: Grids and Sliding-Tile Puzzles

Grid navigation

- ▶ Euclidean distance
- ▶ Manhattan distance

For states (x_1, y_1) and (x_2, y_2) :

$$h_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2|.$$

8-puzzle

- ▶ number of misplaced tiles;
- ▶ sum of Manhattan distances from tiles to their goal positions.

8-puzzle example

Current State

2	8	3
1	6	4
7		5

Misplaced tiles (h_1)

2	8	3
1	6	4
7		5

$h_1 = 6$

Manhattan distance (h_2)

1	2	0
1	1	2
0		1

$h_2 = 8$

Key idea

Different heuristics summarize different approximations of the remaining work.

Common Heuristics Across AI Domains

Traveling Salesperson

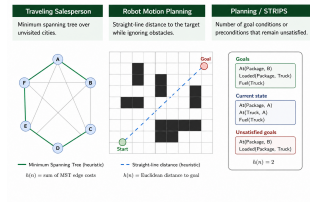
Minimum spanning tree over unvisited cities.

Robot Motion Planning

Straight-line distance to the target while ignoring obstacles.

Planning / STRIPS

Number of goal conditions or preconditions that remain unsatisfied.



Key idea

Heuristic design is problem-specific: the estimate should capture structure that correlates with true remaining cost.

Making Heuristic Functions

1. Solve a relaxed problem

Remove constraints so that the remaining problem is easier to solve.

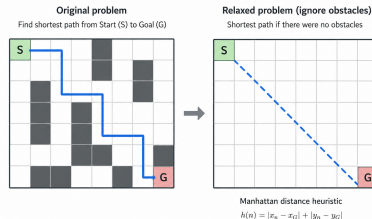
- ▶ ignore obstacles;
- ▶ ignore interactions;
- ▶ ignore selected costs or restrictions.

2. Use domain knowledge

Encode expert knowledge about distance, effort, or utility.

3. Learn the heuristic

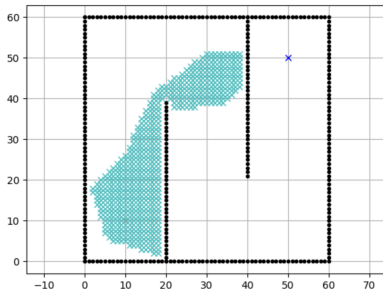
Train a model to approximate cost-to-go from solved instances.



Uninformative Heuristic Regions

An **uninformative region** occurs when many distinct states receive similar or identical heuristic values.

- ▶ The heuristic cannot clearly distinguish which states are more promising.
- ▶ Greedy Best-First Search or A* may expand many states unnecessarily.
- ▶ Search can begin to resemble uninformed exploration.
- ▶ Large flat regions can produce substantial slowdowns.



Key idea

A heuristic is useful only to the extent that its values meaningfully rank competing frontier states.

Part 3: Greedy Best-First Search — Follow the Heuristic



Greedy Best-First Search

Definition

Greedy Best-First Search (GBFS) expands the frontier node with the smallest heuristic value $h(n)$.

$$\text{priority}(n) = h(n)$$

- ▶ GBFS attempts to reach the goal quickly by following the state that appears closest.
- ▶ It repeatedly removes the minimum- h state, expands it, and inserts its successors.
- ▶ It ignores accumulated path cost $g(n)$ when choosing what to expand next.

Key idea

GBFS is greedy because it optimizes only its estimate of the remaining path, not the total solution cost.

Greedy Best-First Search Pseudocode

Algorithm

```
frontier ← PriorityQueue(order by  $h$ )
frontier.insert(initial_state, priority= $h(initial\_state)$ )
came_from[initial_state] ← None

while frontier is not empty:
    state ← frontier.remove_min()
    if goal_test(state):
        return reconstruct_path(came_from, state)
    for child in SUCCESSORS(state):
        if child has not been explored:
            came_from[child] ← state
            frontier.insert(child, priority= $h(child)$ )
return failure
```

Key idea

The priority queue contains only heuristic estimates; edge costs do not affect expansion order.

GBFS Worked Example: From A to Z

Source heuristic values include:

$$h(A) = 14, \quad h(C) = 11, \quad h(E) = 4, \quad h(Z) = 0.$$

Expansion sequence

A : 14



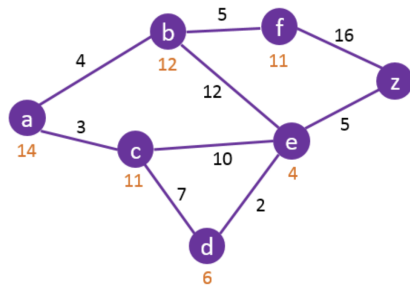
C : 11



E : 4



Z : 0



GBFS chooses the smallest available h value at every step.

Key idea

The route looks promising according to h , even though GBFS does not compare the path costs used to reach those states.

Why Purely Greedy Search Can Be Misleading

GBFS can perform well when the heuristic points reliably toward the goal.

- ▶ It may take long detours because it ignores $g(n)$.
- ▶ A locally attractive state can lead into an expensive region.
- ▶ Weak or deceptive heuristics can cause extensive exploration.
- ▶ GBFS does not provide a general minimum-cost guarantee.

The missing information

Search needs both what we have already paid and what we estimate remains.

Image placeholder

Pathfinding scene where a greedy route is drawn toward the goal but must detour around obstacles.

Part 4: A* Search — Balance Cost-So-Far and Cost-to-Go

A* Combines Two Signals

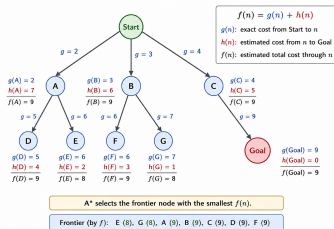
A* evaluates each frontier node using:

$$f(n) = g(n) + h(n).$$

- ▶ $g(n)$: exact path cost from the start to n ;
- ▶ $h(n)$: estimated cost from n to a goal;
- ▶ $f(n)$: estimated total solution cost through n .

Special cases

- ▶ If $h(n) = 0$, A* behaves like Dijkstra / UCS.
- ▶ If $g(n)$ is ignored, the policy becomes GBFS.



Key idea

A* prefers nodes that are inexpensive so far *and* appear inexpensive to finish.

A* Search Pseudocode

Algorithm

```
frontier  $\leftarrow$  PriorityQueue(order by  $f$ )
frontier.insert(start, priority= $h(start)$ )
cost_so_far[start]  $\leftarrow$  0
came_from[start]  $\leftarrow$  None

while frontier is not empty:
    state  $\leftarrow$  frontier.remove_min()
    if goal_test(state): return reconstruct_path(...)
    for child in SUCCESSORS(state):
        new_cost  $\leftarrow g(state) + c(state, child)$ 
        if child is new or new_cost <  $g(child)$ :
             $g(child) \leftarrow$  new_cost
             $f(child) \leftarrow g(child) + h(child)$ 
            frontier.insert(child, priority= $f(child)$ )
```

A* Worked Example: Initial Expansions

Start at A :

$$f(A) = 0 + 14 = 14.$$

Expand A :

$$f(C) = 3 + 11 = 14,$$

$$f(B) = 4 + 12 = 16.$$

Frontier:

$$[C : 14, B : 16].$$

Expand C :

$$f(D) = 10 + 6 = 16, \quad f(E) = 13 + 4 = 17.$$

Image placeholder

A-Z graph with edge costs and heuristic values used by the A* example.

A* Worked Example: Updating the Best Route

Continuing the source example:

- ▶ After expanding B , the frontier contains $D : 16$, $E : 17$, and $F : 20$.
- ▶ Expanding D discovers a cheaper route to E :

$$g(E) = 12, \quad f(E) = 12 + 4 = 16.$$

- ▶ Expanding $E : 16$ generates:

$$f(Z) = 17 + 0 = 17.$$

- ▶ When $Z : 17$ is selected, the search terminates with solution cost 17.

Key idea

Unlike GBFS, A* can revise a promising-looking state when a cheaper path to that state is discovered.

Admissible Heuristics

Definition

A heuristic is **admissible** if it never overestimates the true minimum remaining cost.

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true minimum cost from n to a goal.

- ▶ An admissible heuristic is optimistic: it may underestimate, but it does not claim the goal is farther away than it really is.
- ▶ The closer $h(n)$ is to $h^*(n)$ while remaining admissible, the more strongly it can guide search.

Key idea

Admissibility connects heuristic quality to A*'s minimum-cost guarantee.

Consistent Heuristics

Definition

A heuristic is **consistent** (monotone) if for every transition from n to successor n' :

$$h(n) \leq c(n, n') + h(n').$$

- ▶ The estimated remaining cost cannot decrease by more than the cost of the step just taken.
- ▶ Consistency is a triangle-inequality condition on heuristic estimates.
- ▶ In graph search, consistency supports the standard rule that a state need not be reopened after it has been finalized under the usual A* formulation.

Key idea

Consistency makes heuristic estimates behave coherently along paths, not merely at isolated states.

What A* Pays for Its Guidance

Search costs

- ▶ can store a very large open and closed set;
- ▶ high branching factors still create many candidates;
- ▶ enormous state spaces can remain impractical;
- ▶ priority-queue maintenance adds computation.

Heuristic dependence

- ▶ weak heuristics approach uninformed search;
- ▶ heuristic evaluation itself may be expensive;
- ▶ exact optimal search may be too slow for real-time decisions.

Key idea

A* improves search order, but it does not eliminate the combinatorial growth of difficult state spaces.

Part 5: Variants — Trading Optimality, Memory, and Exploration

Related A* Variants and Tradeoffs

Bidirectional A*

Search simultaneously from the start and goal to reduce the effective search depth.

Iterative Deepening A* (IDA*)

Use heuristic cost thresholds with depth-first traversal to reduce memory requirements.

Beam Search

Keep only a limited number of promising frontier states, sacrificing completeness or optimality for memory and speed.

Weighted A*

Increase the influence of the heuristic to search more aggressively toward the goal.

Weighted A*

Definition

Weighted A* modifies the A* evaluation function by multiplying the heuristic by a weight $w > 1$.

$$f_w(n) = g(n) + w h(n).$$

Weight bias

- ▶ $w = 1$: ordinary A*;
- ▶ larger w : stronger preference for states that appear close to the goal;
- ▶ very large w : behavior approaches GBFS.

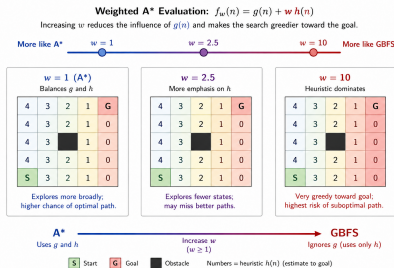
Tradeoff

- ▶ often expands fewer states;
- ▶ may return a suboptimal solution;
- ▶ can provide bounded suboptimality under appropriate heuristic assumptions.

Weighted A* in Uninformative Heuristic Regions

Increasing w makes the search behave more like GBFS.

- ▶ The influence of $g(n)$ becomes smaller relative to $w h(n)$.
- ▶ In a flat heuristic region, many states still receive nearly indistinguishable priorities.
- ▶ The search can lose direction despite using a larger heuristic weight.
- ▶ More aggressive weighting also increases the expected loss of optimality.



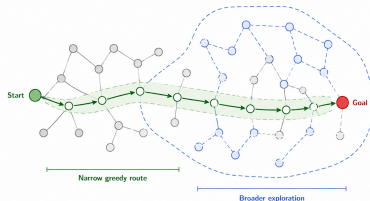
Key idea

Increasing heuristic influence helps only when the heuristic contains useful information to amplify.

Exploration vs. Exploitation in Greedy Search

The source deck introduces mechanisms that deliberately add exploration to greedy search:

- ▶ **Randomized GBFS:** occasionally choose among several best-ranked states.
- ▶ **ϵ -greedy GBFS:** with probability ϵ , choose a random open state; otherwise choose minimum h .
- ▶ **Multiple queues:** maintain frontier views organized by different metadata or search criteria.
- ▶ **Type-based GBFS:** group states into domain-defined types and sample across groups.
- ▶ **Diverse best-first search:** favor dissimilar states among highly ranked candidates.



Exploration Mechanisms in Weighted A*

Randomized focal selection

Break ties or select randomly within a focal list to encourage diversity among similarly promising states.

ε -greedy focal WA*

With probability ε , select a random focal state; otherwise optimize a secondary heuristic.

Type-based Weighted A*

Group focal states by domain-specific type and favor under-explored structural categories.

Key idea

Modern variants can combine bounded-cost search with explicit mechanisms for diversity and exploration.

Search Policies at a Glance

Algorithm	Priority	Main behavior
Dijkstra / UCS	$g(n)$	cheapest path discovered so far
GBFS	$h(n)$	state that appears closest to goal
A*	$g(n) + h(n)$	balance cost-so-far and estimated cost-to-go
Weighted A*	$g(n) + wh(n)$	favor heuristic guidance more aggressively

Key idea

The evaluation function determines what the search algorithm considers “promising.”

Takeaways

- ▶ Uninformed algorithms differ mainly in how they order states using depth or accumulated cost.
- ▶ A heuristic $h(n)$ estimates the remaining cost to a goal and injects domain knowledge into search.
- ▶ GBFS uses only $h(n)$, which can be fast but can also be misled.
- ▶ A* uses $f(n) = g(n) + h(n)$ to combine actual cost and estimated remaining cost.
- ▶ Admissibility and consistency formalize when heuristic guidance is safe for optimal graph search.
- ▶ Weighted and exploratory variants deliberately trade exact optimality, memory, or determinism for practical search performance.

Key idea

Better search comes from matching the frontier priority rule to the information and guarantees the problem actually requires.